

xchan: geometrically encoding a river channel in three dimensions by its cross sections

13 June 2026

Summary

A river channel is a three-dimensional landform, but quantitative descriptions of one reduce to a series of *cross sections* for geomorphology applications: slices taken across the channel at intervals along its length. **xchan** is an R package that takes this reduction seriously as a data structure. It encodes a channel as an ordered collection of cross sections, where each cross section carries a *planimetric* (plan-view) line drawn bank-to-bank on a map and, optionally, a *profile*: the elevation of the ground along that line. Holding both views together gives a lightweight three-dimensional encoding of the channel from which derived geometry can be reconstructed.

From this representation, **xchan** lets users build channels from nominal widths, from arbitrary line segments, from a bankline polygon, or from surveyed and digital-elevation-model (DEM) data; widen (erode) channels by a target width or volume; and compute geometric attributes such as width, elevation, and gradient. The package draws its design from the **sf** package (Pebesma 2018): a single cross section (class **xsection**) is analogous to an **sf** geometry, a channel (class **xchan**) is a list of cross sections analogous to a geometry column, and most functions share an **xt_** prefix mirroring **sf**'s **st_**. It integrates with the wider R spatial stack—**sf**, **terra** (Hijmans 2023), and **centerline** (Tsyplenkov 2025)—and carries physical units throughout via the **units** package (Pebesma, Mailund, and Hiebert 2016).

Statement of need

Cross sections are the common currency of fluvial geomorphology and river engineering, yet the operations surrounding them—generating cross sections along a bankline, attaching surveyed or DEM-derived profiles, eroding banks, and computing width or gradient—are typically assembled from ad hoc GIS steps and one-off scripts. These workflows are slow to reproduce and difficult to embed in a larger analysis.

The need is sharpest when an analysis must be run *repeatedly*. Stochastic models of channel change, for example, rerun an erosion rule many times to build up a distribution of plausible channel evolutions; bank migration and regime analyses likewise sweep over parameters or scenarios (Eaton, Church, and Millar 2010; Church 2006). This is only practical when the channel itself is a first-class, scriptable object that can be created, modified, measured, and re-created programmatically. **xchan** provides exactly this cross-section-first object and a consistent set of verbs that operate on it.

Existing R tools cover neighbouring concerns but not this one. **sf** and **terra** provide excellent general vector and raster primitives, and **centerline** extracts a centerline from a closed polygon, but none offers a channel abstraction that binds plan and profile geometry per cross section or the erosion and channel-summary operations built on it. **xchan** fills that gap. Its intended users are river scientists, geomorphologists, and engineers who work with banklines, channel surveys, and DEMs, and who need channel geometry to be built, modified, and compared across many sections in a reproducible way.

Design and functionality

Encoding a channel by its cross sections

Each cross section bundles a planimetric transect—a bank-to-bank line on the map—with an optional profile. Arrayed along a channel, the planimetric transects alone give a faithful plan view (Figure 1). Keeping the profile optional matters: many tasks (laying out cross sections, measuring planform width, exporting line geometry) need only the plan view, and the profile is attached only when a vertical description is required. Although the two views are treated independently internally—they can be created, flipped, and widened separately—they are bundled into a single **xsection** so that a cross section is a coherent unit. A useful consequence of carrying profiles is that they extend *beyond* the banks into the surrounding floodplain (Figure 2), which is what allows erosion to consume neighbouring topography.

A channel is a list of cross sections

A channel is represented as an ordinary list of **xsection** objects rather than as an opaque “channel” object. This is a deliberate choice: it lets a tabular column of cross-sections live alongside columns of per-section attributes—reach identifiers, roughness, grain size—in a standard data frame or tibble. Channel-level metadata that is genuinely shared (the coordinate reference system, the channel axis, an optional bankline polygon) is stored as attributes on the list and preserved under subsetting. Flow direction and cross-section ordering are governed by the axis, not by list position, so reordering the list does not reorder the channel.

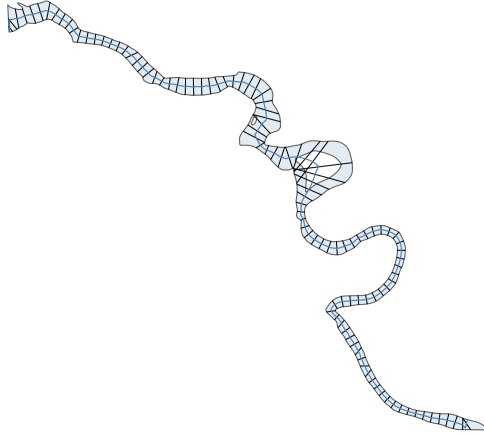


Figure 1: Planimetric cross sections generated along the Squamish River, British Columbia, from a bankline polygon.

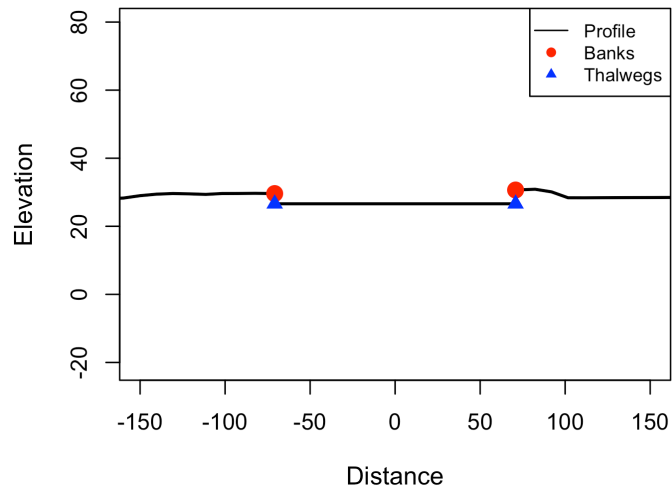


Figure 2: A profile cross section, extending beyond the banks into the surrounding floodplain, with a synthetic 3 m channel dredged into the LiDAR-derived water surface. Bank (red) and thalweg (blue) vertices are marked; the vertical scale is exaggerated 2 times relative to the horizontal so the channel is visible.

Building channels

Channels can be created at several levels of spatial fidelity. `xt_as_channel()` coerces a vector of widths, or a set of line segments, into a channel. `xt_generate_plan()` derives cross sections from a bankline polygon by sampling points along a channel axis (delineated with `centerline` when none is supplied) and spanning bank-to-bank. Profiles are added either from tabular survey data with `xt_add_profile()` or by sampling a DEM along each plan section with `xt_generate_profile()`. In all cases, a channel axis is the authoritative determiner of cross section ordering, not list order.

Erosion by channel widening

Erosion is expressed as channel widening, requested either as a change in width (`dw`) or, when profiles are present, as a change in cross-sectional area (volume per unit length, `dv`); `xt_erosion_width()` and `xt_erosion_volume()` convert between the two. The widening rule is intentionally simple: the left-side channel and the right-side channel are slid outward rigidly, preserving their shape, exactly as if the channel had rectangular walls. Material between the old and new bank positions is removed and the channel bottom is extended at the thalweg elevation. Treating the bank-to-thalweg shape as immaterial during widening keeps the focus on material outside of the banks rather than having to specially decide how much material is taken from the bed.

Specification objects: a recurring design

Several operations need a small piece of configuration that is reused across functions. `xchan` expresses each as a lightweight *specification object* passed as an argument, rather than as a proliferation of similarly named top-level functions:

- **Side** (`side_left()`, `side_right()`, `side_both()`) specifies how widening is allocated between the left and right banks. The allocation is currently a single proportion per bank, but the construction is built to extend naturally to allocations that vary along the channel.
- **Elevation** (`elevation_thalweg()`, `elevation_bank()`, `elevation_bed()`, and variants) specifies *which* elevation a computation should use. The same specification is consumed directly by `xt_elevation()` and reused by functions that need a reference elevation, such as `xt_gradient()` and the dredging tools.
- **Bathymetry** (`bathy_rectangle()`, `bathy_vshape()`) specifies a synthetic channel-bottom shape. DEMs derived from airborne LiDAR typically capture the water surface rather than the submerged bed (Milan et al. 2011), so profiles sampled from them lack bathymetry; `xt_dredge_to()` consumes a bathymetry specification to carve in a plausible channel.

These objects follow the same construction, can be composed (a bathymetry

specification, for instance, takes an elevation specification for its water surface), and provide a single, extensible idiom for referring to channel features. The gradient computation borrows its before/after sliding-window idea from the `slider` package (Vaughan 2024).

Example usage

```
library(xchan)
library(terra)

# Plan-view cross sections every 500 m along a bankline polygon
squamish <- xt_generate_plan(squamish_bankline, spacing = 500)

# Attach profiles by sampling a DEM, then dredge a 3 m channel
# (the LiDAR DEM has no submerged bathymetry)
squamish <- xt_generate_profile(
  squamish,
  unwrap(squamish_dem),
  sample_freq = 10
)
squamish <- xt_dredge_to(
  squamish,
  bathy = bathy_rectangle(depth = 3)
)

# Widen with 70/30 left/right erosion allocation
widened <- xt_widen(squamish, dw = 200, side = side_left(0.7))

# Summaries
widths <- xt_width(widened)
gradient <- xt_gradient(widened, elevation = elevation_bank(min))
```

AI usage disclosure

Generative AI tools were used to assist with drafting package documentation, code and this paper with close instruction by the authors. All AI-assisted output was reviewed, edited, and tested by the authors, who are responsible for the final software and manuscript.

Acknowledgments

The authors thank BGC Engineering Inc. for funding this work and Heba Abdelmoaty for contributing to algorithm design. We also thank the developers

of the `sf`, `terra`, `units`, `centerline`, and `slider` packages, whose tools and design ideas made `xchan` possible.

References

- Church, Michael. 2006. “Bed Material Transport and the Morphology of Alluvial River Channels.” *Annual Review of Earth and Planetary Sciences* 34: 325–54.
- Eaton, Brett C, Michael Church, and Robert G Millar. 2010. “Predicting Downstream Hydraulic Geometry: A Test of Rational Regime Theory.” *Journal of Geophysical Research: Earth Surface* 115 (F4).
- Hijmans, Robert J. 2023. “Terra: Spatial Data Analysis.” *R Package Version 1* (7): 71.
- Milan, David J, George L Heritage, Andrew RG Large, and Neil S Entwistle. 2011. “Terrestrial Laser Scan-Derived Topographic and Roughness Data for Hydraulic Modelling of Gravel-Bed Rivers.” *Earth Surface Processes and Landforms* 36 (10): 1334–48.
- Pebesma, Edzer. 2018. “Simple Features for r: Standardized Support for Spatial Vector Data.” *The R Journal* 10 (1): 439–46.
- Pebesma, Edzer, Thomas Mailund, and James Hiebert. 2016. “Measurement Units in R.” *The R Journal* 8 (2): 486–94. <https://doi.org/10.32614/RJ-2016-061>.
- Tsyplenkov, Anatoly. 2025. *Centerline: Extract Centerline from Closed Polygons*. <https://CRAN.R-project.org/package=centerline>.
- Vaughan, Davis. 2024. *Slider: Sliding Window Functions*. <https://CRAN.R-project.org/package=slider>.